

LAPORAN APLIKASI NILAI EIGEN DAN EIGENFACE PADA PENGENALAN WAJAH



Dosen Pengampu: Drs. Bambang Harjito, M.App.Sc, PhD



ANGGOTA KELOMPOK:

FAUSTINA HELENA AFNY A.	L0125040
TSANIYA NURFADHILAH	L0125117
ALYAA NADHIIFAH W.	L0125141

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA
UNIVERSITAS SEBELAS MARET**

2026

DAFTAR ISI

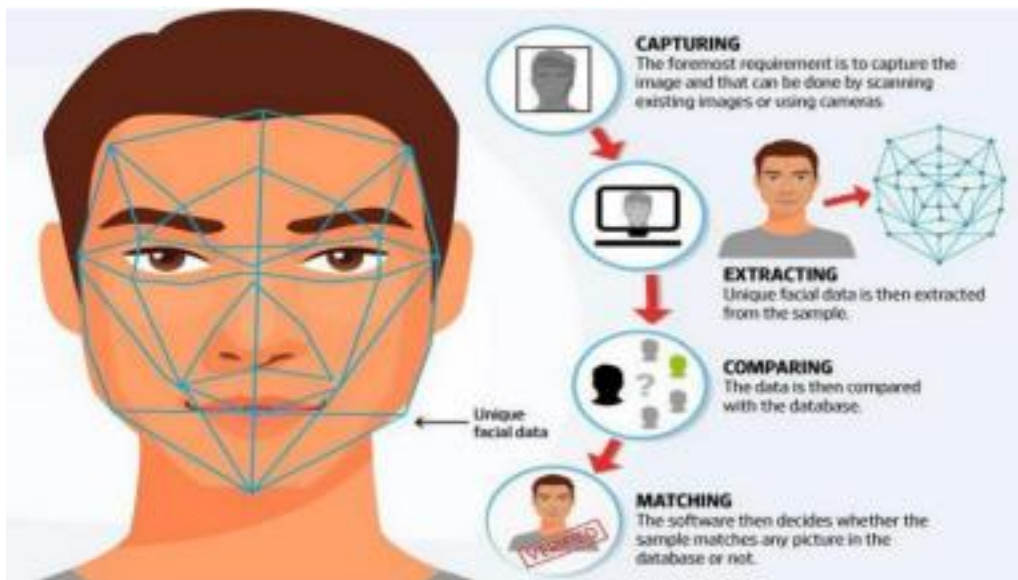
DAFTAR ISI	2
BAB I	4
A. DESKRIPSI UMUM TUGAS PROJECT PEMBUATAN APLIKASI	4
B. SPESIFIKASI TUGAS	6
BAB II.....	8
BAB III	10
BAB IV	11
BAB V	13
PENUTUP	13
LAMPIRAN	14

BAB I

DESKRIPSI MASALAH

A. DESKRIPSI UMUM TUGAS PROJECT PEMBUATAN APLIKASI

Pengenalan wajah (Face Recognition) adalah teknologi biometrik yang bisa dipakai untuk mengidentifikasi wajah seseorang untuk berbagai kepentingan khususnya keamanan. Program pengenalan wajah melibatkan kumpulan citra wajah yang sudah disimpan pada database lalu berdasarkan kumpulan citra wajah tersebut, program dapat mempelajari bentuk wajah lalu mencocokkan antara kumpulan citra wajah yang sudah dipelajari dengan citra yang akan diidentifikasi. Alur proses sebuah sistem pengenalan wajah diperlihatkan pada Gambar 1.

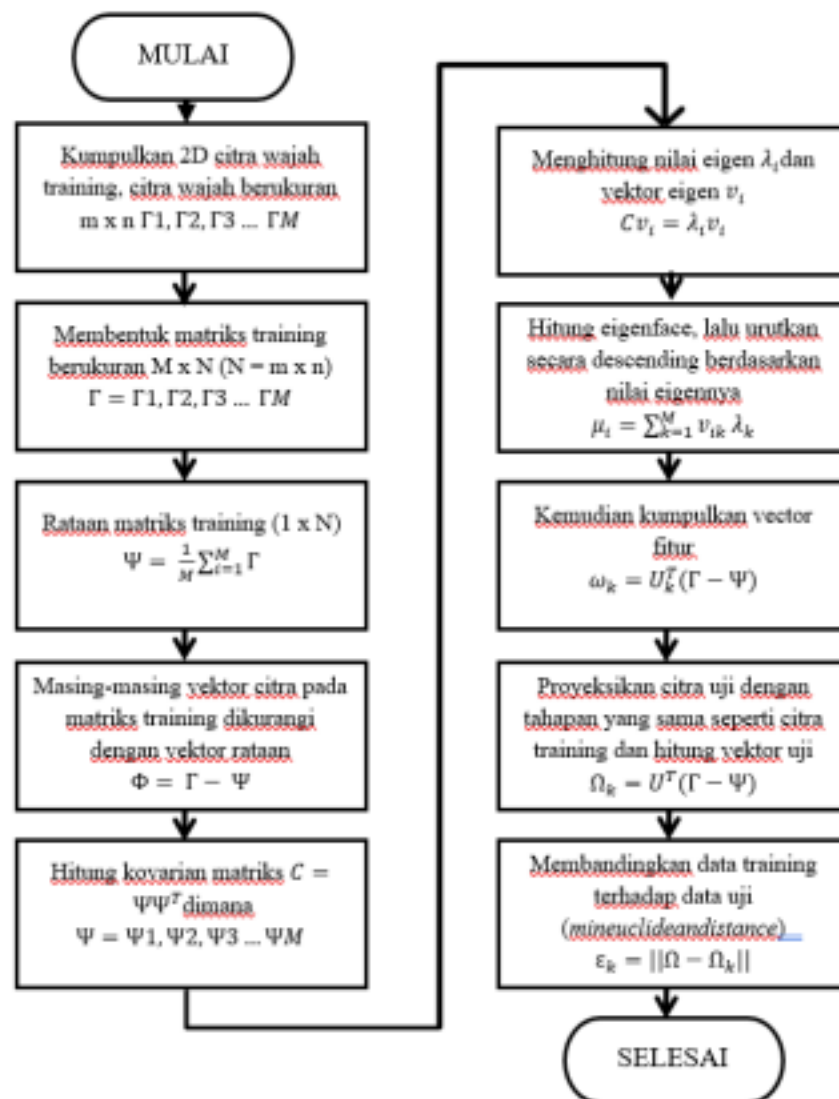


Terdapat berbagai teknik untuk memeriksa citra wajah dari kumpulan citra yang sudah diketahui seperti jarak Euclidean dan cosine similarity, principal component analysis (PCA), serta Eigenface. Pada Tugas ini, akan dibuat sebuah program pengenalan wajah menggunakan Eigenface.

Sekumpulan citra wajah akan digunakan dengan representasi matriks. Dari representasi matriks tersebut akan dihitung sebuah matriks Eigenface. Program pengenalan wajah dapat dibagi menjadi 2 tahap berbeda yaitu tahap training dan pencocokkan. Pada tahap training,

akan diberikan kumpulan data set berupa citra wajah. Citra wajah tersebut akan dinormalisasi dari RGB ke Grayscale (matriks), hasil normalisasi akan digunakan dalam perhitungan eigenface. Seperti namanya, matriks eigenface menggunakan eigenvector dalam pembentukannya. Berikut merupakan langkah rinci dalam pembentukan eigenface.

Flow chart Algoritma eigen face



Pada tahapan akhir, akan ditemui gambar dengan euclidean distance paling kecil maka gambar tersebut yang dikenali oleh program paling menyerupai test face selama nilai

kemiripan di bawah suatu nilai batas. Jika nilai minimum di atas nilai batas maka dapat dikatakan tidak terdapat citra wajah yang mirip dengan test face.

Program dibuat dengan Bahasa Python dengan memanfaatkan sejumlah library di OpenCV (Computer Vision) atau library pemrosesan gambar lainnya (contoh PIL). Fungsi untuk mengekstraksi fitur dari sebuah citra wajah tidak perlu anda buat lagi, tetapi menggunakan fungsi ekstraksi yang sudah tersedia di dalam library. Fungsi Eigen dilarang import dari library dan harus diimplementasikan, sedangkan untuk operasi matriks lainnya silahkan menggunakan library.

Kode program untuk ekstraksi fitur dapat dibaca pada artikel ini: Feature extraction and similar image search with OpenCV for newbies, pada laman:

<https://medium.com/machine-learning-world/feature-extraction-and-similar-image-search-with-opencv-for-newbies-3c59796bf774>

Nilai batas kemiripan citra test face dapat ditentukan oleh pembuat program melalui percobaan.

Berikut merupakan referensi pengenalan wajah dengan metode eigenface:

<https://jurnal.untan.ac.id/index.php/jcskommipa/article/download/9727/9500>

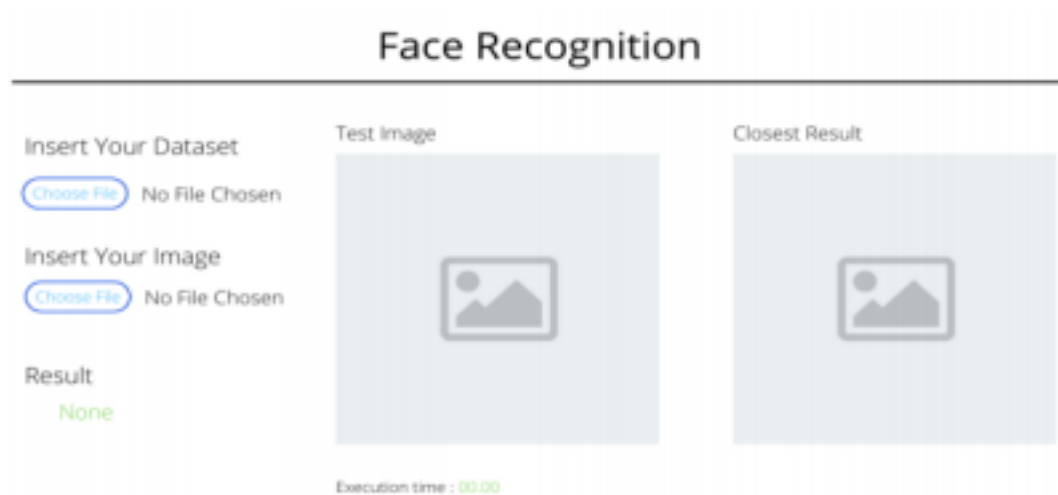
<https://www.geeksforgeeks.org/ml-face-recognition-using-eigenfaces-pca-algorithm>

PENGUNAAN PROGRAM

Berikut ini adalah input yang akan dimasukkan pengguna untuk eksekusi pr

1. **Folder dataset**, berisi folder atau directory yang berisi kumpulan gambar yang digunakan sebagai training image.
2. **File gambar**, berisi file gambar input yang ingin dikenali dengan format file yang bebas selama merupakan format untuk gambar.

Tampilan layout dari aplikasi web yang akan dibangun kurang lebih adalah sebagai berikut. Anda dapat mengubah layout selama layout masih terdiri dari komponen yang sama.



Catatan: Warna biru menunjukkan komponen yang dapat di klik. Warna hijau menunjukkan luaran yang didapat dari hasil eksekusi. Anda dapat menambahkan menu lainnya, gambar, logo, dan sebagainya. Tampilan GUI dibuat semenarik mungkin selama mencakup seluruh informasi pada layout yang diberikan di atas. Kreativitas menjadi salah satu komponen penilaian.

B. SPESIFIKASI TUGAS

Buatlah program pengenalan wajah dalam bahasa Python berbasis GUI dengan spesifikasi sebagai berikut:

1. Program menerima input folder dataset dan sebuah gambar citra wajah.
2. Basis data wajah dapat diunduh secara mandiri melalui <https://www.kaggle.com/datasets/hereisburak/pins-face-recognition>
3. Program menampilkan gambar citra wajah yang dipilih oleh pengguna.
4. Program melakukan pencocokan wajah dengan koleksi wajah yang ada di folder yang telah dipilih. Metrik untuk pengukuran kemiripan menggunakan eigenface + jarak euclidean.
5. Program menampilkan 1 hasil pencocokan pada dataset yang paling dekat dengan gambar input atau memberikan pesan jika tidak didapatkan hasil yang sesuai.

6. Program menghitung jarak euclidean dan nilai eigen & vektor eigen yang ditulis sendiri. Tidak boleh menggunakan fungsi yang sudah tersedia di dalam library atau Bahasa Python.

BAB II

DASAR TEORI

Sistem pengenalan wajah otomatis merupakan salah satu bidang studi yang paling dinamis dan penuh tantangan dalam dunia visi komputer serta interaksi antara manusia dan komputer. Secara historis, upaya formal pertama untuk mengklasifikasikan ciri-ciri wajah manusia dikemukakan oleh Francis Galton pada tahun 1888. Akan tetapi, penelitian di bidang ini mengalami masa vakum yang cukup panjang sepanjang dasawarsa 1980-an. Kebangkitan yang signifikan terjadi ketika para peneliti mulai menyadari bahwa wajah tidak harus dianalisis berdasarkan rekonstruksi geometri tiga dimensi yang kompleks, melainkan dapat diperlakukan sebagai masalah pengenalan pola dua dimensi dengan memanfaatkan keuntungan dari posisi wajah yang umumnya dalam keadaan tegak.

Kendala utama dalam pemrosesan citra digital wajah terletak pada dimensi datanya yang sangat besar. Sebuah citra wajah digital berukuran standar, misalnya 100×100 piksel, direpresentasikan sebagai sebuah titik dalam ruang vektor berdimensi 10.000. Menjalankan algoritma klasifikasi secara langsung pada ruang berdimensi tinggi ini tidak hanya membutuhkan sumber daya komputasi yang sangat besar, tetapi juga rentan terhadap fenomena *curse of dimensionality* yang menyebabkan kegagalan generalisasi atau *overfitting*.

Untuk mengatasi kendala dimensi tersebut, dikembangkanlah sebuah metode reduksi dimensi berbasis statistik yang dikenal dengan nama *Principal Component Analysis* (PCA). Dalam konteks visi komputer, penerapan PCA pada kumpulan data citra wajah menghasilkan sekumpulan basis ortogonal yang disebut *Eigenfaces*. Pendekatan representasi yang efisien ini pertama kali dipelajari oleh Sirovich dan Kirby, kemudian dimanfaatkan secara luas oleh Matthew Turk dan Alex Pentland untuk keperluan klasifikasi dan identifikasi wajah secara praktis. Metode ini memproyeksikan citra wajah asli dari ruang piksel berdimensi tinggi ke dalam ruang bagian (*subspace*) berdimensi rendah yang disebut sebagai "ruang wajah" (*face space*). Melalui transformasi ini, informasi penting dari wajah dapat dipertahankan sementara komponen informasi yang berulang atau tidak esensial dihilangkan.

Landasan matematis metode *eigenface* bertumpu pada konsep dekomposisi nilai dan vektor eigen dari matriks kovarians yang dibangun menggunakan sekumpulan citra latih. Proses pembentukan ruang eigen ini dapat diuraikan melalui langkah-langkah sistematis berikut:

Misalkan terdapat sebuah dataset pelatihan yang terdiri dari M buah citra wajah. Agar analisis statistik berjalan konsisten, seluruh citra harus melalui tahap pra-pemrosesan berupa penyelarasan posisi fitur wajah utama (seperti mata dan mulut), penyesuaian ukuran piksel menjadi resolusi standar $r \times c$, serta penerapan teknik simetrisasi untuk mengatasi variasi pencahayaan yang buruk. Setiap citra kemudian dikonversi menjadi vektor kolom tunggal $\Gamma_i \in \mathbb{R}^D$, dengan dimensi total $D = r \times c$.

Sebelum menghitung matriks kovarians, nilai rata-rata dari seluruh vektor wajah dalam *dataset* harus dihitung terlebih dahulu menggunakan persamaan:

$$\Psi = \frac{1}{M} \sum_{i=1}^M \Gamma_i$$

Setiap vektor citra wajah asli kemudian dikurangi dengan citra rata-rata Ψ untuk menghasilkan vektor deviasi atau vektor terpusat Φ_i :

$$\Phi_i = \Gamma_i - \Psi$$

Tahap pemusatan data ini sangat penting dalam analisis komponen utama karena berfungsi untuk menghilangkan korelasi palsu dalam *dataset*. Dengan memusatkan data pada titik asal koordinat, algoritma dapat berkonsentrasi penuh pada variasi struktur wajah antarindividu, tanpa terpengaruh oleh tingkat kecerahan rata-rata citra secara keseluruhan.

Kumpulan citra latih asli disusun berdampingan sebagai kolom-kolom di dalam matriks *dataset* $\mathbf{T}$ yang berdimensi $D \times M$:

$$\mathbf{T} = [\Gamma_1 \Gamma_2 \dots \Gamma_M]_{D \times M} = \begin{bmatrix} t_{1,1} & t_{1,2} & \dots & t_{1,M} \\ t_{2,1} & t_{2,2} & \dots & t_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ t_{D,1} & t_{D,2} & \dots & t_{D,M} \end{bmatrix}$$

Di mana setiap kolom merupakan satu buah wajah yang telah diratakan (*flattened*) dari matriks gambar dua dimensi menjadi vektor satu dimensi sepanjang D piksel.

Setelah menghitung vektor citra rata-rata $\Psi \in \mathbb{R}^{D \times 1}$, kita membentuk matriks deviasi $\mathbf{A}$ dengan dimensi yang sama dengan $\mathbf{T}$, yaitu $D \times M$:

$$\mathbf{A} = [\Phi_1 \Phi_2 \dots \Phi_M]_{D \times M} = \begin{bmatrix} (\Gamma_{1,1} - \Psi_1) & (\Gamma_{1,2} - \Psi_1) & \dots & (\Gamma_{1,M} - \Psi_1) \\ (\Gamma_{2,1} - \Psi_2) & (\Gamma_{2,2} - \Psi_2) & \dots & (\Gamma_{2,M} - \Psi_2) \\ \vdots & \vdots & \ddots & \vdots \\ (\Gamma_{D,1} - \Psi_D) & (\Gamma_{D,2} - \Psi_D) & \dots & (\Gamma_{D,M} - \Psi_D) \end{bmatrix}$$

Matriks $\mathbf{A}$ ini merepresentasikan variasi murni dari setiap piksel wajah terhadap rata-rata wajah global.

Matriks kovarians $\mathbf{C} \in \mathbb{R}^{D \times D}$ dari kumpulan data wajah terpusat didefinisikan sebagai hasil kali luar (*outer product*) matriks deviasi:

$$\mathbf{C} = \frac{1}{M} \sum_{i=1}^M \Phi_i \Phi_i^T = \frac{1}{M} \mathbf{A} \mathbf{A}^T$$

Operasi perkalian ini dapat divisualisasikan sebagai berikut:

$$\mathbf{C} = \frac{1}{M} \begin{bmatrix} \Phi_{1,1} & \dots & \Phi_{1,M} \\ \vdots & \ddots & \vdots \\ \Phi_{D,1} & \dots & \Phi_{D,M} \end{bmatrix}_{D \times M} \times \begin{bmatrix} \Phi_{1,1} & \dots & \Phi_{D,1} \\ \vdots & \ddots & \vdots \\ \Phi_{1,M} & \dots & \Phi_{D,M} \end{bmatrix}_{M \times D}$$

Dekomposisi nilai eigen langsung pada matriks kovarians $\mathbf{C}$ bertujuan untuk mencari vektor eigen $\mathbf{u}_k$ dan nilai eigen λ_k yang memenuhi persamaan karakteristik:

$$\mathbf{C} \mathbf{u}_k = \lambda_k \mathbf{u}_k$$

Dekomposisi nilai eigen langsung pada matriks kovarians $\mathbf{C}$ bertujuan untuk mencari vektor eigen $\mathbf{u}_k$ dan nilai eigen λ_k yang memenuhi persamaan karakteristik:

$$\mathbf{C} \mathbf{u}_k = \lambda_k \mathbf{u}_k$$

Namun, karena dimensi piksel D umumnya sangat besar (misalnya 10.000 elemen untuk citra beresolusi 100×100), matriks $\mathbf{C}$ akan berukuran sangat masif yaitu 10.000×10.000 dengan total 10^8 elemen. Menyelesaikan persamaan karakteristik pada matriks dengan skala sebesar ini secara langsung tidak layak dilakukan karena keterbatasan memori dan daya komputasi.

Untuk mengatasi kendala komputasi ini, Turk dan Pentland menerapkan teorema aljabar linear yang dikenal sebagai *Covariance Matrix Trick*. Alih-alih menghitung vektor eigen dari matriks $\mathbf{A}\mathbf{A}^T$ yang berukuran $D \times D$, perhitungan dialihkan ke matriks antara $\mathbf{L} = \mathbf{A}^T\mathbf{A}$ yang hanya berukuran $M \times M$, di mana jumlah citra pelatihan jauh lebih sedikit daripada jumlah piksel ($M \ll D$).

Operasi perkalian dalam matriks $\mathbf{L}$ ini didefinisikan sebagai:

$$\mathbf{L} = \mathbf{A}^T\mathbf{A} = \begin{bmatrix} \Phi_{1,1} & \dots & \Phi_{D,1} \\ \vdots & \ddots & \vdots \\ \Phi_{1,M} & \dots & \Phi_{D,M} \end{bmatrix}_{M \times D} \times \begin{bmatrix} \Phi_{1,1} & \dots & \Phi_{1,M} \\ \vdots & \ddots & \vdots \\ \Phi_{D,1} & \dots & \Phi_{D,M} \end{bmatrix}_{D \times M}$$

Setiap elemen $L_{i,j}$ dari matriks $\mathbf{L}$ berdimensi $M \times M$ ini sebenarnya adalah hasil kali dalam (*inner product*) antara deviasi citra ke- i dengan citra ke- j :

$$L_{i,j} = \Phi_i^T \Phi_j$$

Persamaan nilai eigen untuk matriks $\mathbf{L}$ dinyatakan sebagai:

$$\mathbf{A}^T\mathbf{A}\mathbf{v}_k = \mu_k \mathbf{v}_k$$

Jika kedua ruas dari persamaan di atas dikalikan dari arah kiri dengan matriks deviasi $\mathbf{A}$, maka diperoleh formulasi sebagai berikut:

$$\mathbf{A}(\mathbf{A}^T\mathbf{A}\mathbf{v}_k) = \mathbf{A}(\mu_k \mathbf{v}_k)$$

$$(\mathbf{A}\mathbf{A}^T)(\mathbf{A}\mathbf{v}_k) = \mu_k (\mathbf{A}\mathbf{v}_k)$$

Persamaan di atas membuktikan secara matematis bahwa vektor $\mathbf{u}_k = \mathbf{A}\mathbf{v}_k$ merupakan vektor eigen dari matriks kovarians asli $\mathbf{C} = \mathbf{A}\mathbf{A}^T$. Selain itu, nilai eigen λ_k untuk matriks kovarians asli bernilai identik dengan nilai eigen μ_k dari matriks dimensi rendah $\mathbf{L}$.

Dalam implementasi pemrograman, vektor eigen $\mathbf{u}_k$ ini harus dinormalisasi agar memenuhi syarat basis ortonormal ($\|\mathbf{u}_k\| = 1$). Melalui pendekatan teoretis ini, analisis komponen utama pada citra berdimensi tinggi dapat diselesaikan dengan efisiensi waktu komputasi yang sangat tinggi karena kita hanya melakukan dekomposisi pada matriks $\mathbf{L}$ yang berukuran kecil.

Setiap citra wajah baru yang dimasukkan ke dalam sistem ($\mathbf{\Gamma}_{\text{test}}$) harus melewati tahap pemusatan data terlebih dahulu dengan mengurangi citra rata-rata pelatihan ($\mathbf{\Phi}_{\text{test}} = \mathbf{\Gamma}_{\text{test}} - \mathbf{\Psi}$). Vektor deviasi ini kemudian diproyeksikan ke dalam ruang wajah dengan melakukan perkalian titik terhadap basis *eigenfaces* terpilih:

$$w_k = \mathbf{u}_k^T \mathbf{\Phi}_{\text{test}}$$

Hasil dari proyeksi ini adalah vektor bobot berdimensi rendah yang merepresentasikan koordinat wajah tersebut dalam ruang eigen:

$$\mathbf{\Omega}_{\text{test}} = [w_1, w_2, \dots, w_k]^T$$

Proses identifikasi dilakukan dengan membandingkan vektor bobot citra uji $\mathbf{\Omega}_{\text{test}}$ terhadap seluruh vektor bobot citra referensi yang telah disimpan sebelumnya di galeri sistem ($\mathbf{\Omega}_i$, di mana $i = 1, 2, \dots, M$). Pengukuran tingkat kemiripan diselesaikan dengan menghitung jarak Euclidean minimum menggunakan pengklasifikasi tetangga terdekat (*1-Nearest Neighbor*):

$$\epsilon_i = \|\mathbf{\Omega}_{\text{test}} - \mathbf{\Omega}_i\|$$

Sistem mengidentifikasi citra uji sebagai individu i apabila nilai jarak ϵ_i merupakan nilai terkecil dari seluruh perbandingan di dalam database.

Kerangka kerja pengenalan wajah berbasis *eigenface* tidak hanya mampu mencocokkan wajah yang telah dikenal, tetapi juga dilengkapi kemampuan cerdas untuk menolak citra non-wajah serta mengenali wajah baru yang belum terdaftar dalam basis data. Logika ini dibangun

dengan menghitung jarak rekonstruksi θ_c , yaitu jarak antara vektor citra uji terpusat Φ_{test} dengan proyeksi rekonstruksinya kembali pada ruang wajah Φ_f :

$$\Phi_f = \sum_{j=1}^k w_j \mathbf{u}_j$$

$$\theta_c^2 = \|\Phi_{\text{test}} - \Phi_f\|^2$$

BAB III

IMPLEMENTASI

1.1 Gambaran Umum

Face Recognition System adalah aplikasi *desktop* berbasis Python yang mengimplementasikan metode pengenalan wajah menggunakan algoritma Eigenface dan Principal Component Analysis (PCA). Program ini terdiri dari tiga file utama yang bekerja secara modular dan terpisah, yaitu `backend.py` (logika komputasi), `frontend.py` (antarmuka pengguna), dan `visualisasi.py` (jendela visualisasi perhitungan matematis).

Sistem bekerja dengan cara melatih model dari kumpulan gambar wajah dalam folder dataset, lalu mengenali wajah baru berdasarkan jarak Euclidean pada ruang eigenface yang telah terbentuk. Pengguna dapat menyesuaikan ambang batas threshold untuk mengontrol sensitivitas pengenalan.

1.2 Arsitektur Sistem

Arsitektur: 3 file terpisah, `backend.py` (komputasi), `frontend.py` (UI), `visualisasi.py` (grafik). Tidak ada ketergantungan sirkular.

backend.py	frontend.py	visualisasi.py
Komputasi PCA & Eigenface	GUI CustomTkinter & Webcam	Grafik Matplotlib 5 Tab
<i>import backend</i>	<i>import backend + visualisasi</i>	<i>(standalone)</i>

1.3 Alur Kerja Program

- 1.1. Pengguna memilih *folder dataset* (berisi *subfolder* per identitas, masing-masing berisi foto wajah)

- 1.2. *Backend* membaca semua gambar dengan `os.walk()`, mengonversi ke *grayscale*, *resize* ke 100×100 piksel, lalu *flatten* menjadi vektor 1D
- 1.3. SVD dihitung pada matriks wajah terpusat (dikurangi *mean face*) untuk mendapatkan *eigenfaces*
- 1.4. Pengguna mengunggah foto uji atau mengambil foto via *webcam*
- 1.5. Sistem memproyeksikan foto uji ke ruang *eigenface* dan menghitung jarak Euclidean ke seluruh bobot *dataset*
- 1.6. Identitas dengan jarak minimum dikembalikan jika di bawah *threshold*; jika tidak, wajah dinyatakan Unknown
- 1.7. Tombol Lihat Visualisasi membuka jendela 5 *tab* dengan grafik interaktif hasil perhitungan

1.4 Fitur Utama

- *Dataset input*: pilih *folder* utama dengan banyak *subfolder* (*multi-class*), otomatis dilatih ulang
- *Input uji*: *upload* gambar *file* (JPG/PNG) atau *capture* langsung dari *webcam* (~30 fps)
- *Threshold slider*: rentang 1.000–10.000, mengontrol sensitivitas pengenalan secara *real-time*
- Visualisasi interaktif 5 *tab*: *Eigenface grid*, Grafik Nilai Eigen, Visualisasi $Ax = \lambda x$, Plot Vektor *Eigenface* Pertama, Evaluasi Akurasi LOOCV + *Confusion Matrix*
- Status *training*: menampilkan jumlah gambar yang berhasil dimuat dan proses *training*

Dokumentasi Program (Penjelasan Kode)

2.1 File: backend.py

File ini berisi seluruh logika komputasi pengenalan wajah. Tidak mengimpor modul UI apapun sehingga dapat digunakan mandiri atau diuji terpisah.

2.1.1 Class FaceRecognitionBackend

Deskripsi: Kelas tunggal yang merangkum proses pelatihan (*train*) dan pengenalan wajah (*recognize*). Menggunakan SVD sebagai implementasi PCA tanpa *library scikit-learn*

Atribut Instance:

Atribut	Tipe	Fungsi
dataset_path	str	Path folder root dataset yang dipilih pengguna
size	tuple (int,int)	Ukuran gambar yang seragam, default (100,100) piksel
image_names	list[str]	Daftar label identitas (nama subfolder) tiap gambar latih
mean_face	ndarray (10000,)	Rata-rata semua gambar latih, wajah rata-rata
eigenfaces	ndarray (10000, N)	Matriks eigenface hasil SVD, kolom = tiap eigenface
weights	ndarray (N, K)	Proyeksi tiap gambar latih ke ruang eigenface

Metode:

Nama	Tipe/Return	Keterangan
__init__()	None	Inisialisasi atribut dengan nilai default kosong
train()	bool	Baca dataset, hitung mean face, jalankan SVD, simpan eigenfaces & weights. Return True jika berhasil

_project_and_match()	tuple	Proyeksi vektor gambar ke eigenspace, hitung jarak Euclidean ke seluruh weights, return nama + jarak + akurasi
recognize(path)	tuple (4)	Baca gambar dari file path, panggil _project_and_match, return nama, jarak, akurasi, waktu komputasi
recognize_from_frame()	tuple (4)	Terima array BGR dari webcam (tanpa menulis file), konversi ke grayscale, panggil _project_and_match

2.1.2 Detail Implementasi train()

```

for root, dirs, files in os.walk(self.dataset_path):
    for file in files:
        if file.lower().endswith(('.jpg', '.png', '.jpeg')):
            path = os.path.join(root, file)
            img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
            if img is not None:
                img = cv2.resize(img, self.size)
                images.append(img.flatten())
                # Gunakan nama subfolder sebagai label identitas
                folder_name = os.path.basename(root)
                self.image_names.append(folder_name)

    if len(images) == 0:
        return False

```

```

faces = np.array(images)

self.mean_face = np.mean(faces, axis=0)

centered_faces = faces - self.mean_face

# Singular Value Decomposition (SVD) untuk mencari Eigenfaces

U, S, Vt = np.linalg.svd(centered_faces.T, full_matrices=False)

self.eigenfaces = U

self.weights = np.dot(self.eigenfaces.T, centered_faces.T).T

```

os.walk() digunakan agar sistem dapat membaca *dataset* bertingkat (*folder* per identitas). Label identitas diambil dari nama *subfolder* (os.path.basename(root)), bukan dari nama *file* gambar. SVD dilakukan pada matriks transpos sehingga *eigenface* didapat langsung sebagai kolom matriks U.

2.1.3 Detail Implementasi _project_and_match()

```

"""Shared projection + matching logic."""
centered_test = gray_flat - self.mean_face
test_weight = np.dot(self.eigenfaces.T, centered_test)
distances = np.linalg.norm(self.weights - test_weight, axis=1)
best_match_idx = np.argmin(distances)
min_distance = distances[best_match_idx]

if min_distance > threshold:
    return "Unknown", min_distance, 0.0

accuracy = (1 - (min_distance / threshold)) * 100

```

Gambar uji diproyeksikan ke ruang *eigenface* yang sama dengan data latih. Jarak Euclidean dihitung secara *batch* (*vectorized*) menggunakan np.linalg.norm dengan axis=1, efisien karena tidak menggunakan *loop* Python. Persentase akurasi bersifat relatif: semakin kecil jarak dibanding *threshold*, semakin tinggi akurasi.

2.2 File: frontend.py

File utama yang dijalankan pengguna. Mengimpor FaceRecognitionBackend dan VisualisasiWindow. Terdiri dari dua *class*: WebcamWindow (jendela kamera) dan FaceApp (aplikasi utama).

2.2.1 Class WebcamWindow

Deskripsi: Jendela *pop-up* (CTkToplevel) yang menampilkan *live feed* kamera dan tombol Capture. Kamera berjalan di *thread* terpisah agar UI tidak *freeze*.

Atribut	Tipe	Fungsi
<code>_cap</code>	<code>cv2.VideoCapture</code>	Objek kamera OpenCV, dibuka di <code>_start_camera()</code>
<code>_running</code>	<code>bool</code>	Flag kontrol loop kamera di thread daemon
<code>_last_frame</code>	<code>ndarray BGR</code>	Frame terakhir yang berhasil dibaca dari kamera
<code>on_capture_callback</code>	<code>callable</code>	Fungsi callback dari FaceApp yang dipanggil setelah foto diambil

Nama	Tipe/Return	Keterangan
<code>_start_camera()</code>	<code>None</code>	Buka <code>VideoCapture(0)</code> , validasi, set <code>_running=True</code> , jalankan thread <code>_feed_loop</code> sebagai daemon
<code>_feed_loop()</code>	<code>None</code>	Loop membaca frame ~30fps, konversi <code>BGR→RGB→CTkImage</code> , jadwalkan update UI via <code>self.after()</code>
<code>_update_feed()</code>	<code>None</code>	Update label gambar di main thread (dipanggil via <code>after()</code> , thread-safe)
<code>_capture()</code>	<code>None</code>	Salin frame terakhir, tutup kamera, panggil <code>on_capture_callback(frame)</code>

<code>_close()</code>	None	Set <code>_running=False</code> , release kamera, destroy window
-----------------------	------	--

```
def _feed_loop(self):
    while self._running:
        ret, frame = self._cap.read()
        if not ret:
            break
        self._last_frame = frame
        # Convert BGR → RGB → PIL → CTkImage
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        pil_img = Image.fromarray(rgb)
        ctk_img = ctk.CTkImage(pil_img, size=(520, 390))
        # Schedule UI update on main thread
        self.after(0, self._update_feed, ctk_img)
        # ~30 fps
        import time; time.sleep(0.033)
```

Pembaruan UI dilakukan menggunakan `self.after(0, callback)` agar aman dijalankan dari *thread non-main*. OpenCV membaca gambar dalam format BGR, sehingga harus dikonversi ke RGB sebelum ditampilkan dengan PIL/CTkImage.

2.2.2 Class FaceApp

Deskripsi: Aplikasi utama (CTk window). Mengelola sidebar konfigurasi (kiri) dan workspace hasil (kanan). Mengorkestrasi komunikasi antara UI, backend, webcam, dan jendela visualisasi.

Atribut	Tipe	Fungsi
backend	FaceRecognitionBackend	Instance backend, satu-satunya titik akses ke logika komputasi

<code>test_img_path</code>	<code>str</code> <code>None</code>	Path file gambar uji yang dipilih pengguna (<code>None</code> jika webcam)
<code>_webcam_frame</code>	<code>ndarray</code> <code>None</code>	Frame BGR hasil capture webcam (<code>None</code> jika input dari file)
<code>_result_img</code>	<code>CTkImage</code>	Referensi permanen gambar hasil agar tidak di-garbage-collect oleh Python

Nama	Tipe/Return	Keterangan
<code>setup_layout()</code>	<code>None</code>	Bangun seluruh UI: sidebar (dataset, upload, threshold, tombol) dan workspace (panel gambar, status bar)
<code>browse_dataset()</code>	<code>None</code>	Buka dialog folder, set <code>backend.dataset_path</code> , panggil <code>backend.train()</code> , tampilkan status
<code>browse_file()</code>	<code>None</code>	Buka dialog file gambar, tampilkan preview di panel kiri, reset webcam frame
<code>on_webcam_toggle()</code>	<code>None</code>	Saat checkbox webcam dicentang: reset file yang terupload (mutually exclusive)
<code>open_webcam_window()</code>	<code>None</code>	Buat WebcamWindow dengan callback <code>_on_webcam_capture</code>
<code>_on_webcam_capture()</code>	<code>None</code>	Terima frame dari WebcamWindow, tampilkan di panel kiri, langsung jalankan <code>_do_recognition()</code>
<code>run_recognition()</code>	<code>None</code>	Validasi dataset & input, lalu cabangkan ke webcam atau file
<code>_do_recognition()</code>	<code>None</code>	Panggil backend sesuai mode input, tampilkan hasil di panel kanan & status bar
<code>open_visualisasi()</code>	<code>None</code>	Validasi dataset sudah dimuat, buat <code>VisualisasiWindow</code> dan <code>grab_set</code> untuk fokus

<code>clear_uploaded_file()</code>	None	Reset semua state input: path, frame, label, gambar panel
<code>on_slider_move()</code>	None	Update label nilai threshold secara real-time saat slider digeser

2.2.3 Detail `_do_recognition()`, Logika Inti UI

```
def _do_recognition(self):
    threshold = int(self.slider_thresh.get())

    if self._webcam_frame is not None:
        name, dist, acc, comp_time = self.backend.recognize_from_frame(
            self._webcam_frame, threshold=threshold
        )
    else:
        name, dist, acc, comp_time = self.backend.recognize(
            self.test_img_path, threshold=threshold
        )

    if name is None:
        messagebox.showwarning("Warning", "Gagal memproses perhitungan pengenalan wajah.")
        return

    self.lbl_metrics.configure(
        text=f"Accuracy Level: {acc:.2f}% | Computation Time: {comp_time:.4f} second"
    )

    if name == "Unknown":
        self.lbl_img_result.configure(image=None,
                                       text="⚠ Wajah Tidak Dikenal\n(Melebihi Threshold)")
        self.lbl_tag_result.configure(text="Identified Face: Unknown")
    else:
        # Cari gambar referensi di subfolder yang namanya == name
        matched_path = None
```

```

for root, dirs, files in os.walk(self.backend.dataset_path):
    if os.path.basename(root) == name:
        for f in files:
            if f.lower().endswith(('.jpg', '.png', '.jpeg')):
                matched_path = os.path.join(root, f)
                break
        if matched_path:
            break # ← break di sini, LALU display di luar loop

# Tampilkan gambar — di LUAR for loop
if matched_path:
    img_match = Image.open(matched_path)
    self._result_img = ctk.CTkImage(img_match, size=(340, 340)) # simpan di self!
    self.lbl_img_result.configure(image=self._result_img, text="")

```

Kunci penting: gambar hasil disimpan di `self._result_img` (*atribut instance*), bukan variabel lokal. Jika disimpan sebagai variabel lokal, Python *garbage collector* akan menghapusnya setelah *method* selesai dan gambar tidak akan tampil di label.

2.3 File: visualisasi.py

File terpisah yang tidak mengubah logika *backend* maupun *frontend*. Membuka jendela `CTkToplevel` berisi 5 *tab* visualisasi hasil perhitungan *eigenface*. Seluruh komputasi (termasuk LOOCV) dilakukan di dalam *file* ini menggunakan data yang sudah tersedia di objek *backend*.

2.3.1 Fungsi Helper (Module-level)

Nama	Tipe/Return	Keterangan
<code>_style(ax, fig)</code>	None	Terapkan tema gelap (background hitam, teks putih, grid abu) ke axes matplotlib
<code>_embed(fig, master)</code>	FigureCanvasTkAgg	Embed Figure matplotlib ke dalam <code>tk.Frame</code> , widget dikonfigurasi transparan agar tidak hitam

<code>_wrap(ctlk_tab)</code>	<code>tk.Frame</code>	Bungkus CTKTab dengan <code>tk.Frame</code> biasa; solusi wajib agar <code>canvas matplotlib</code> tidak tampil hitam di dalam CTKTabview
------------------------------	-----------------------	--

2.3.2 Class VisualisasiWindow

Deskripsi: CTKToplevel dengan 5 *tab* interaktif. Menerima objek *backend* dan *threshold* awal dari FaceApp. *Slider threshold* di *header* memungkinkan *refresh* Tab 2 (Nilai Eigen) dan Tab 5 (Akurasi) secara *real-time* tanpa menutup jendela.

Atribut	Tipe	Fungsi
<code>backend</code>	FaceRecognitionBackend	Referensi ke backend yang sudah dilatih, hanya dibaca, tidak dimodifikasi
<code>threshold</code>	<code>int</code>	Nilai ambang batas aktif, diperbarui saat slider digeser
<code>_tab1-5</code>	<code>tk.Frame</code>	Frame pembungkus tiap tab (hasil <code>_wrap</code>), diisi oleh method <code>_build_*</code>
<code>_ev_fig</code>	Figure	Figure matplotlib khusus Tab Visualisasi Vektor Eigen (interaktif slider λ)
<code>_ev_canvas</code>	FigureCanvasTkAgg	Canvas tab tersebut, di-redraw saat <code>_on_ev_slide</code> dipanggil
<code>_ev_norm</code>	<code>ndarray</code>	Eigenvalue ternormalisasi dari dataset, digunakan untuk label kasus di 4-panel

Nama	Tipe/Return	Keterangan
<code>_build()</code>	None	Bangun header, slider threshold, CTKTabview 5 tab, panggil semua <code>_build_*</code> method
<code>_on_thr(val)</code>	None	Update <code>self.threshold</code> , hapus & rebuild Tab Nilai Eigen dan Tab Akurasi

<code>_build_eigenface()</code>	None	Tab 1: Render mean face + 9 eigenface teratas sebagai grid gambar grayscale
<code>_build_norm_graph()</code>	None	Tab 2: Bar+line chart nilai eigen (S^2) dengan garis threshold dan highlight komponen dominan
<code>_build_eigen_vector_vis()</code>	None	Tab 3: Diagram $Ax=\lambda x$, vektor x dan 4 kasus λ dalam satu plot koordinat
<code>_build_vector_plot()</code>	None	Tab 4: Area chart oranye nilai tiap piksel dari eigenface pertama (vektor 10.000 elemen)
<code>_build_accuracy()</code>	None	Tab 5: LOOCV, hitung confusion matrix, tampilkan heatmap biru + persentase akurasi
<code>_on_ev_slide(val)</code>	None	Update label λ dan keterangan kasus, panggil <code>_draw_eigen_fig(lam)</code>
<code>_draw_eigen_fig(lam)</code>	None	Gambar ulang Figure 2-panel: kiri=koordinat $Ax=\lambda x$, kanan=4 kasus dengan kasus aktif di-highlight

2.3.3 Cara Kerja LOOCV di Tab Evaluasi Akurasi

```

for i in range(n):
    mask = np.ones(n, dtype=bool)
    mask[i] = False
    dists = np.linalg.norm(weights[mask] - weights[i], axis=1)
    bidx = np.argmin(dists)
    mdist = dists[bidx]
    train_names = [names[j] for j in range(n) if j != i]
    y_true.append(names[i])
    y_pred.append(train_names[bidx] if mdist <= self.threshold else "Unknown")

correct = sum(t == p for t, p in zip(y_true, y_pred))
accuracy = correct / n * 100

```

Leave-One-Out Cross Validation (LOOCV) dilakukan sepenuhnya dalam `visualisasi.py` menggunakan *weights* yang sudah tersimpan di *backend*. Setiap gambar secara bergantian menjadi data uji sementara sisanya menjadi data latih. Ini tidak memerlukan pelatihan ulang karena *weights* sudah tersedia, hanya perlu menghitung jarak di *eigenspace*.

2.3.4 Solusi Layer Hitam pada CTkTabview

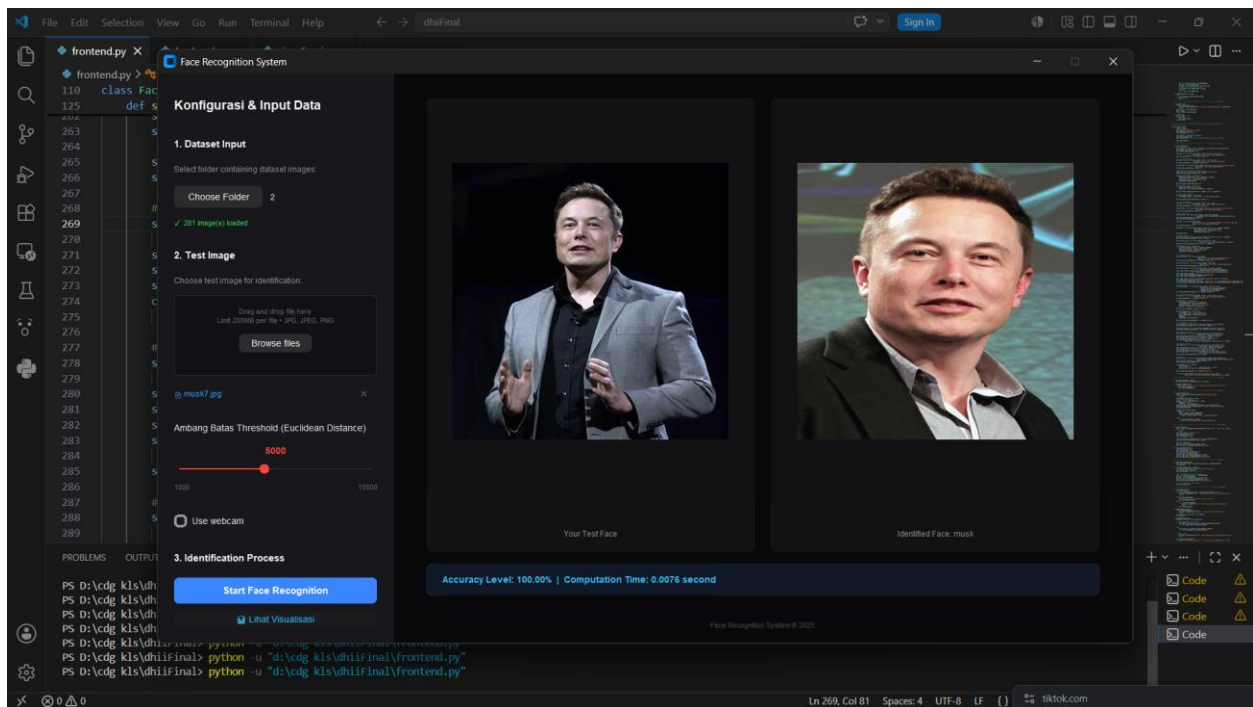
```
def _wrap(ctl_tab):  
    """tk.Frame di dalam CTkTab agar canvas tidak hitam."""  
    ctl_tab.configure(fg_color=_PANEL)  
    f = tk.Frame(ctl_tab, bg=_BG)  
    f.pack(fill="both", expand=True)  
    return f
```

CTkTabview menggunakan layer warna internal yang menimpa *background* matplotlib. Solusinya: bungkus setiap *tab* dengan `tk.Frame` polos (bukan `CTkFrame`) sebelum meletakkan matplotlib. Ini adalah *work-around* yang dikenal di komunitas `customtkinter`.

Analisis Program saat Dijalankan

3.1 Antarmuka Utama (FaceApp)

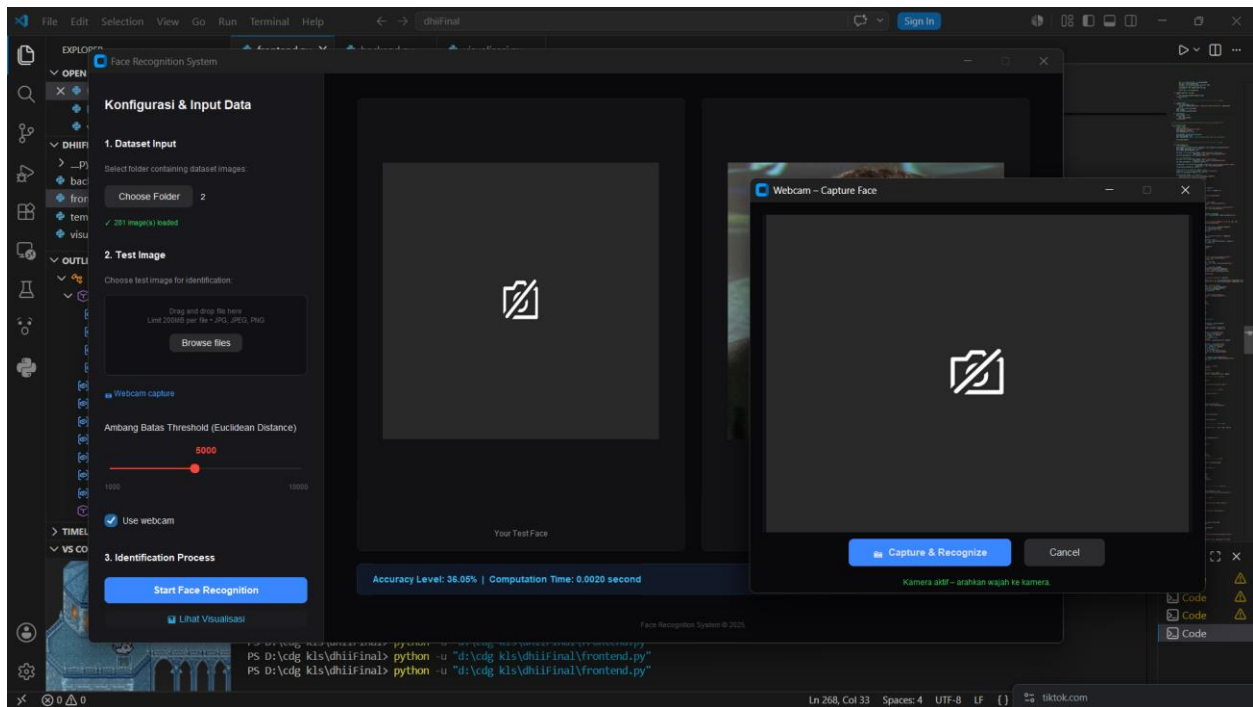
Saat program dijalankan, jendela 1200×700 piksel muncul dengan *layout* dua panel: *sidebar* konfigurasi (290px, kiri) dan *workspace* hasil (kanan).



Komponen UI	Perilaku & Analisis
Choose Folder	Membuka dialog folder sistem. Setelah dipilih, backend.train() dipanggil sinkron di main thread, UI menampilkan label 'Training...' kuning lalu berubah hijau '✓ N image(s) loaded' jika berhasil, atau merah jika tidak ada gambar valid
Threshold Slider	Rentang 1.000–10.000 Euclidean Distance. Nilai kecil = sistem sangat ketat (banyak Unknown); nilai besar = sistem lebih permisif (lebih banyak wajah dikenali, termasuk kemungkinan false positive)
Browse files	Membuka file dialog. Gambar langsung dipreview di panel kiri (340×340 px). Memilih file otomatis unchecked checkbox webcam
Use webcam	Saat dicentang + Start ditekan: membuka WebcamWindow. Capture otomatis menjalankan pengenalan dan hasilnya langsung muncul di panel kanan
Start Face Recognition	Validasi dataset dan input, lalu cabang ke recognize() atau recognize_from_frame() berdasarkan sumber input
Panel kiri / kanan	Panel kiri menampilkan gambar uji, panel kanan menampilkan foto pertama dari folder identitas yang cocok. Keduanya 340×340 px dengan CTkImage (bukan ImageTk) agar tidak di-garbage-collect
Status bar bawah	Menampilkan Accuracy Level (%) dan Computation Time (detik, 4 desimal) setelah pengenalan selesai

Lihat Visualisasi

Disabled jika eigenfaces belum ada. Membuka VisualisasiWindow sebagai modal (grab_set) dengan threshold awal 5000



3.2 Analisis Proses Training

Saat *folder dataset* dipilih, *backend* melakukan `os.walk()` dan memproses setiap gambar. Berikut analisis kompleksitas dan dampaknya:

- Waktu *training*: $O(N \times D)$ di mana N = jumlah gambar, D = dimensi (10.000 piksel). SVD adalah *bottleneck* utama dengan kompleksitas $O(\min(N,D)^2 \times \max(N,D))$
- Memori: setiap gambar membutuhkan $\sim 80\text{KB}$ ($10.000 \times \text{float64}$). *Dataset* 300 gambar $\approx 24\text{MB}$ RAM untuk matriks *faces*
- Label: diambil dari nama *subfolder*, bukan nama *file*, sehingga satu *folder* boleh berisi banyak foto untuk satu identitas
- Jika gambar tidak dapat dibaca (*corrupt*/format tidak didukung): `cv2.imread()` mengembalikan `None` dan gambar dilewati tanpa *error*

3.3 Analisis Proses Pengenalan

Pengenalan wajah dilakukan dalam 3 langkah utama:

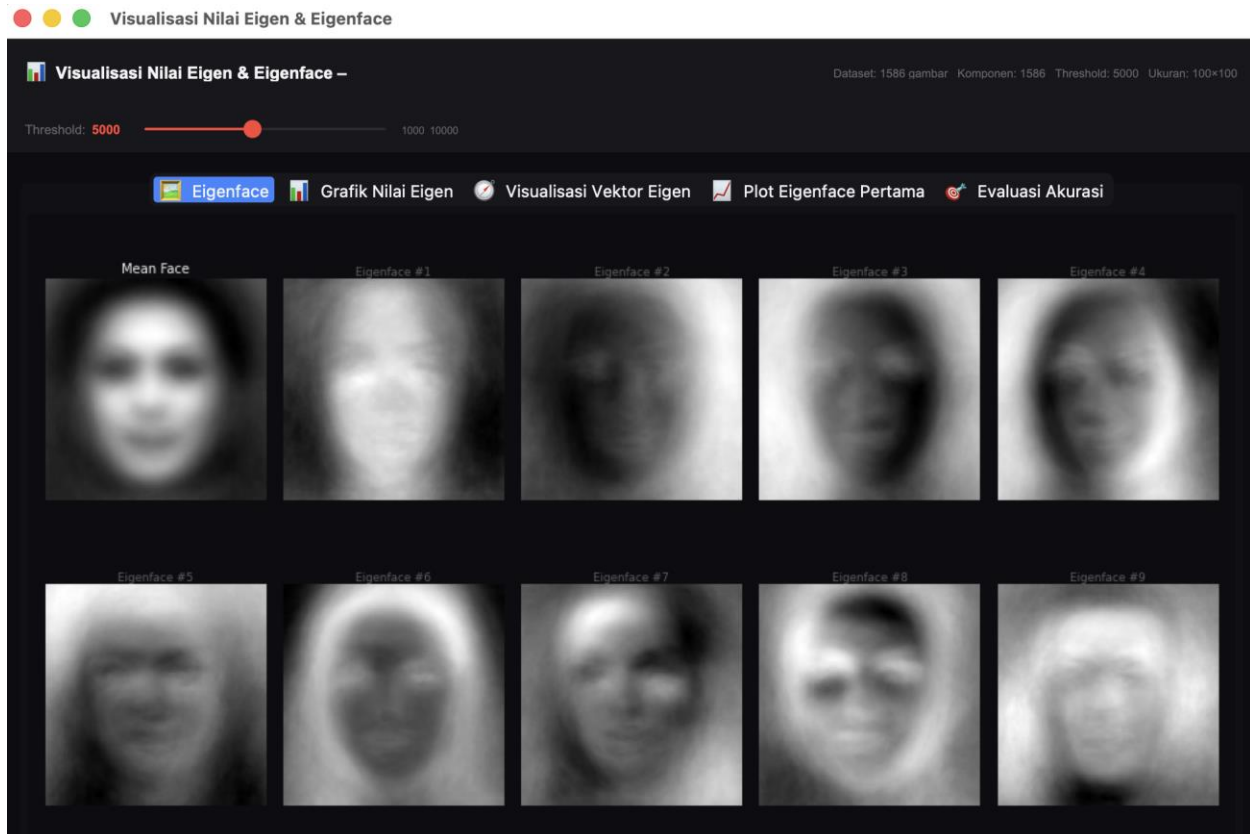
1. Proyeksi: gambar uji dikurangi `mean_face` lalu dikalikan `eigenfaces.T`, menghasilkan vektor `test_weight` berdimensi N komponen
2. Jarak: `np.linalg.norm(weights - test_weight, axis=1)` menghitung jarak Euclidean ke seluruh N gambar latih sekaligus (*vectorized*, tanpa *loop*)
3. Keputusan: jika $\text{min_distance} \leq \text{threshold} \rightarrow$ dikenali dengan akurasi = $(1 - \text{dist}/\text{threshold}) \times 100\%$; jika tidak $\rightarrow$ Unknown

Waktu komputasi biasanya di bawah 0.05 detik bahkan untuk *dataset* ratusan gambar karena seluruh operasi dilakukan secara *numpy vectorized*.

3.4 Analisis Hasil Visualisasi 5 Tab

Tab 1 Eigenface Grid

Menampilkan *mean face* (rata-rata semua gambar latih) dan 9 *eigenface* teratas. *Mean face* terlihat sebagai wajah 'generik' tanpa fitur spesifik. *Eigenface* pertama menangkap variasi terbesar (paling informatif), sedangkan *eigenface* ke-9+ menangkap variasi halus seperti bayangan atau ekspresi minor.



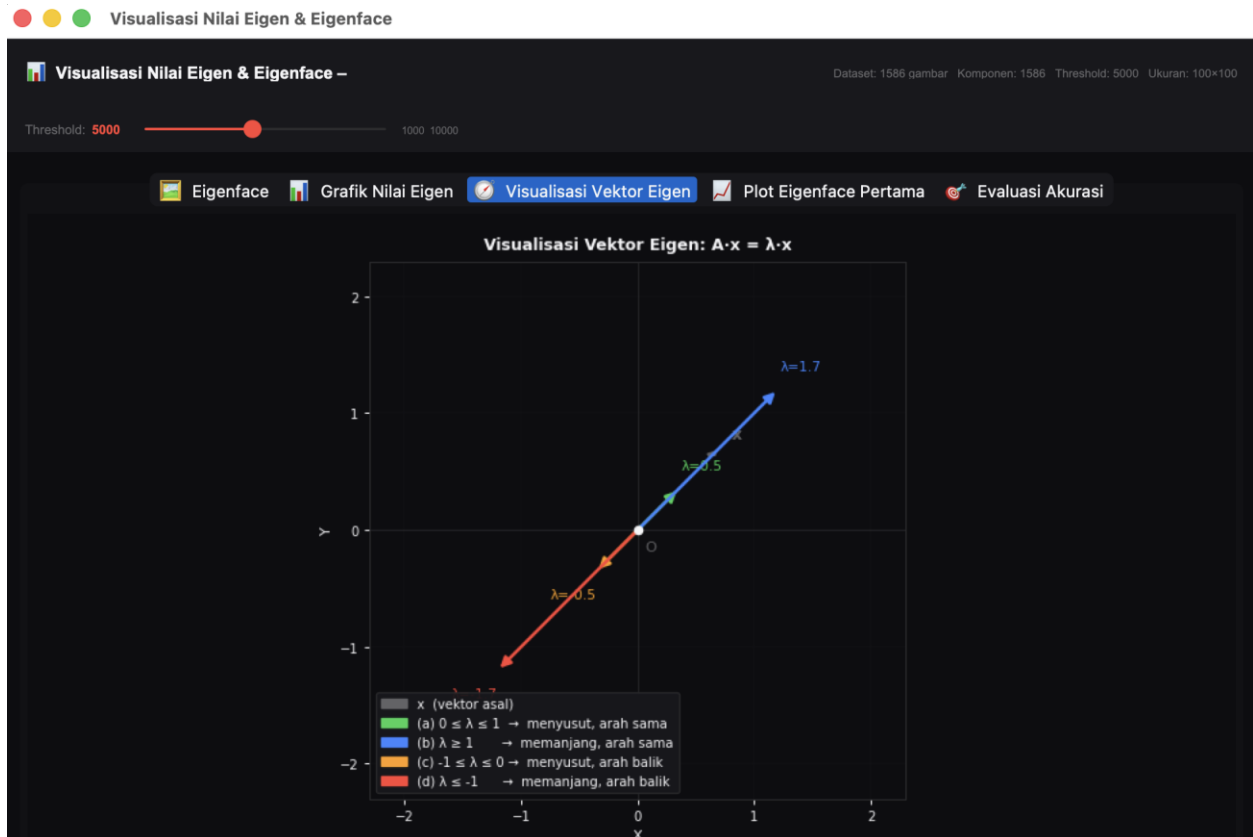
Tab 2 Grafik Nilai Eigen

Bar *chart* nilai eigen (S^2) dari SVD. Komponen awal memiliki nilai jauh lebih besar dari komponen berikutnya, ini adalah ciri khas PCA yang berarti sebagian besar informasi wajah terkandung dalam sedikit *eigenface* pertama. Area biru menunjukkan komponen yang nilai eigennya melebihi *threshold*².



Tab 3 Visualisasi $Ax=\lambda x$

Plot konsep matematis vektor eigen berdasarkan *slide* kuliah. Menampilkan 4 kasus efek λ terhadap vektor x : menyusut sama arah ($0 \leq \lambda \leq 1$), memanjang sama arah ($\lambda \geq 1$), menyusut balik arah ($-1 \leq \lambda \leq 0$), memanjang balik arah ($\lambda \leq -1$). *Slider* λ di bawah memungkinkan eksplorasi interaktif, kasus aktif di-*highlight* dengan border emas.



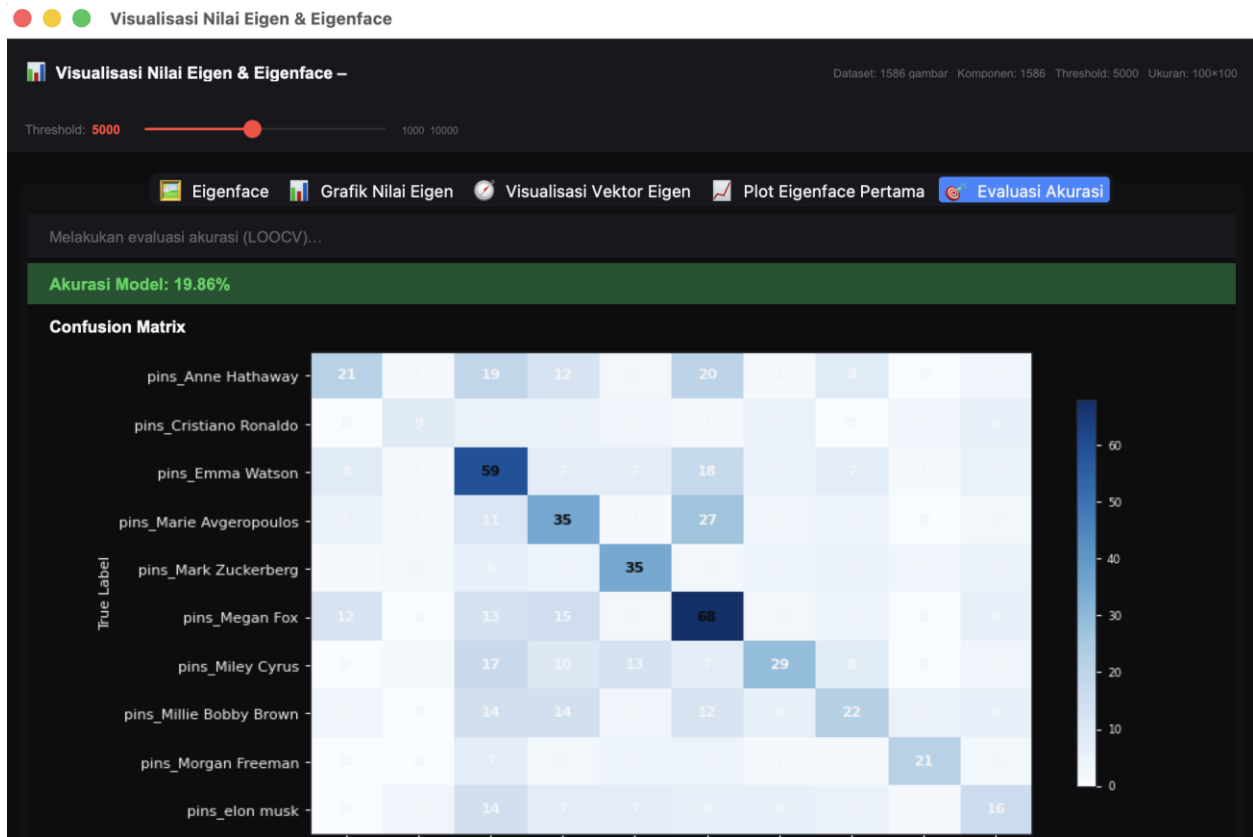
Tab 4 Plot Vektor Eigenface Pertama

Area *chart* oranye menampilkan nilai bobot tiap piksel dari *eigenface* ke-1 (vektor 10.000 elemen untuk gambar 100×100). Pola gelombang yang terlihat merepresentasikan variasi intensitas piksel yang paling membedakan antar wajah dalam *dataset*. Nilai mendekati nol berarti piksel tersebut tidak terlalu informatif untuk perbedaan.



Tab 5 Evaluasi Akurasi (LOOCV + Confusion Matrix)

Leave-One-Out Cross Validation mengevaluasi model secara *fair*: setiap gambar dijadikan data uji sekali. *Confusion matrix heatmap* menunjukkan distribusi prediksi, diagonal utama (cerah) = prediksi benar, *off-diagonal* (gelap) = salah klasifikasi. *Slider threshold* dapat digeser untuk melihat dampaknya terhadap akurasi, *threshold* lebih tinggi umumnya meningkatkan *recall*, tetapi menurunkan *precision*.



3.5 Analisis Pengaruh Threshold

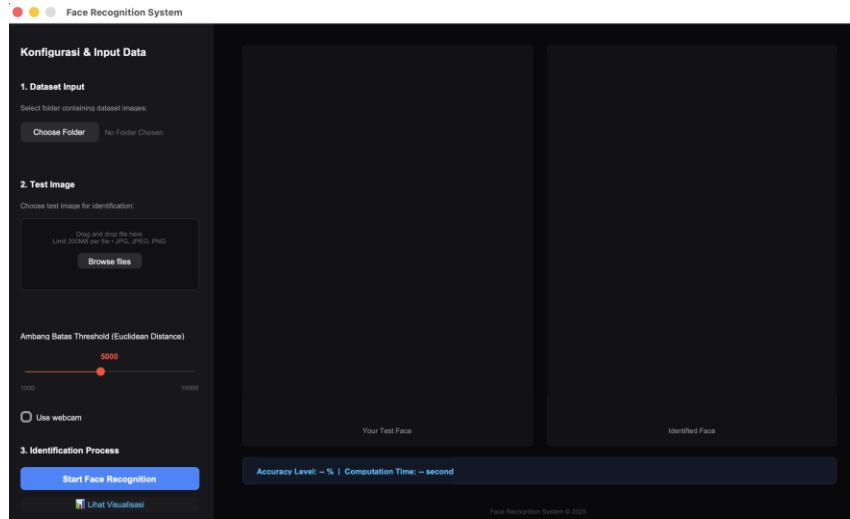
Threshold	Dampak pada Dataset Internal	Dampak pada Dataset Eksternal	Akurasi LOOCV
< 3.000	Banyak gambar sendiri tidak dikenali (false negative tinggi)	Aman, hampir semua eksternal ditolak	Rendah (~30%)
5.000 (default)	Gambar identik (jarak 0.00) pasti dikenali	Wajah sangat berbeda (kucing, dll.) ditolak	~31–50%
7.000	Sebagian besar wajah mirip dikenali	Wajah manusia mirip bisa salah diklasifikasi	~70–80%
> 8.000	Semua hampir dikenali (recall tinggi)	Risiko false positive tinggi, kucing bisa dikenali sebagai manusia	~80%+ (bias)

3.6 Kelebihan dan Keterbatasan Sistem

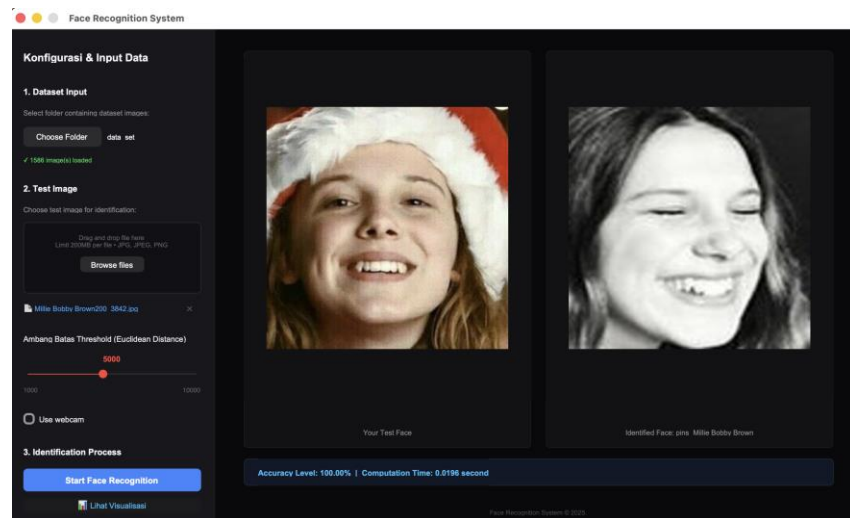
Kelebihan	Keterbatasan
• Tidak bergantung scikit-learn, implementasi SVD mandiri dengan NumPy • Dataset multi-class: subfolder per identitas, jumlah kelas tidak dibatasi • Pengenalan vectorized, cepat bahkan untuk ratusan gambar • Webcam real-time dengan threading, UI tidak freeze • Visualisasi interaktif lengkap dengan LOOCV terintegrasi	• Sensitif terhadap pencahayaan dan pose, tidak ada preprocessing alignment • Training sinkron di main thread, UI dapat terasa lambat untuk dataset besar (>1000 gambar) • Threshold manual, tidak ada kalibrasi otomatis per dataset • Tidak ada face detection, gambar harus sudah berisi wajah saja • Akurasi terbatas pada ~31% (threshold default), meningkat signifikan dengan threshold lebih tinggi namun berisiko false positive

BAB IV

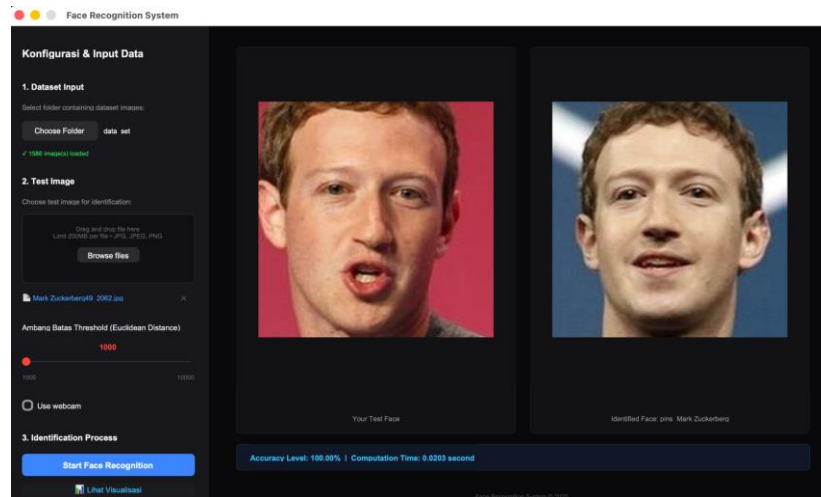
HASIL PERCOBAAN



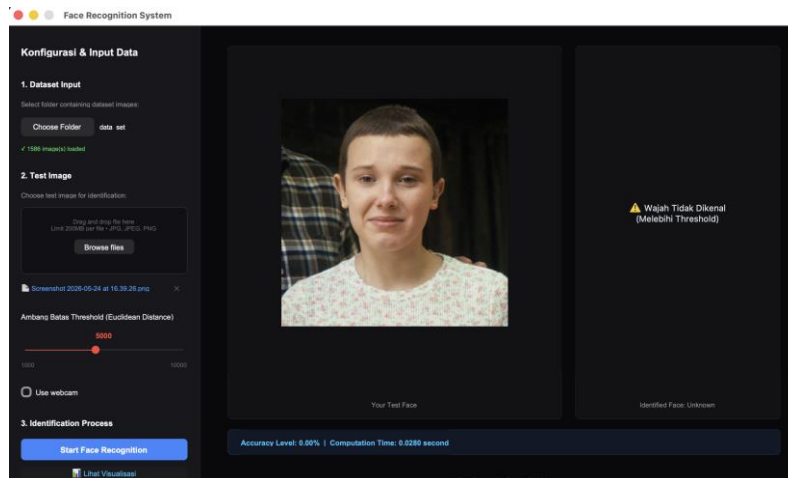
4.1. Tampilan awal aplikasi *face recognition*



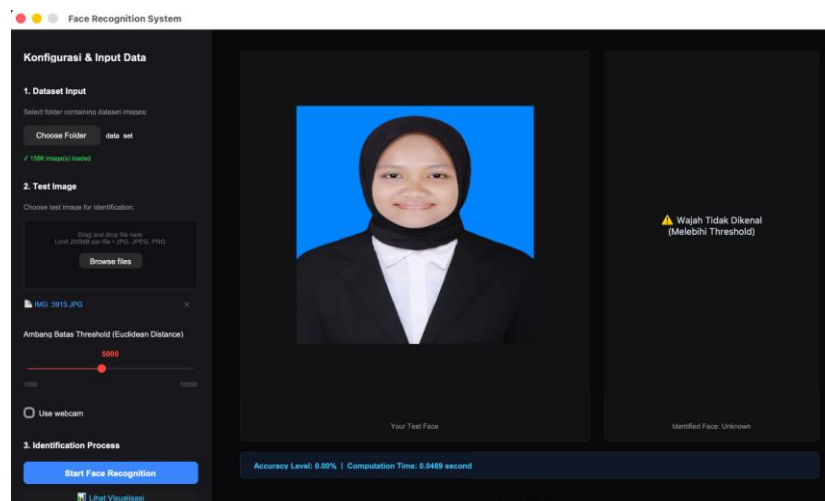
4.2. Tampilan aplikasi *face recognition* ketika gambar uji terdapat *folder dataset*



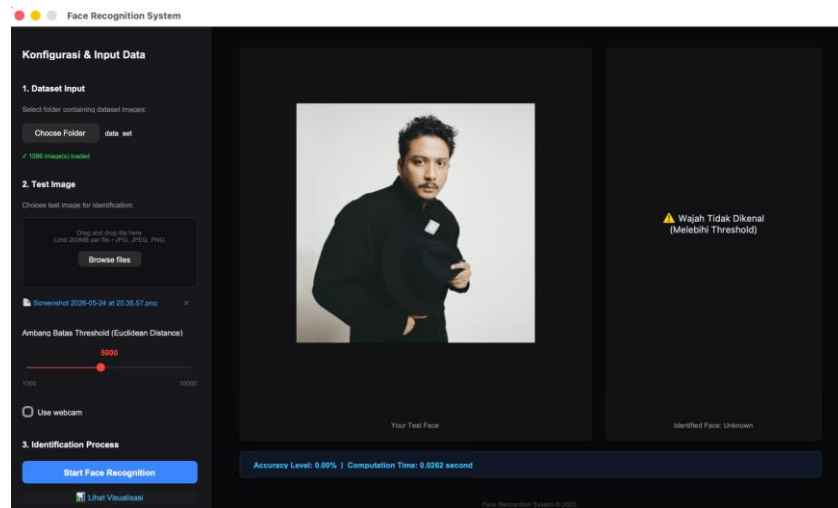
4.3. Tampilan aplikasi *face recognition* ketika gambar uji terdapat *folder dataset*



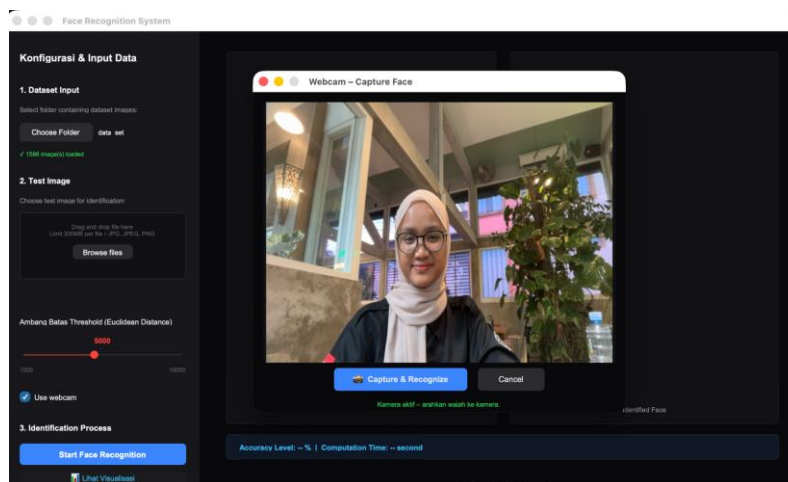
4.4. Tampilan aplikasi *face recognition* ketika gambar uji tidak ditemukan dalam *folder dataset* dengan individu yang sama



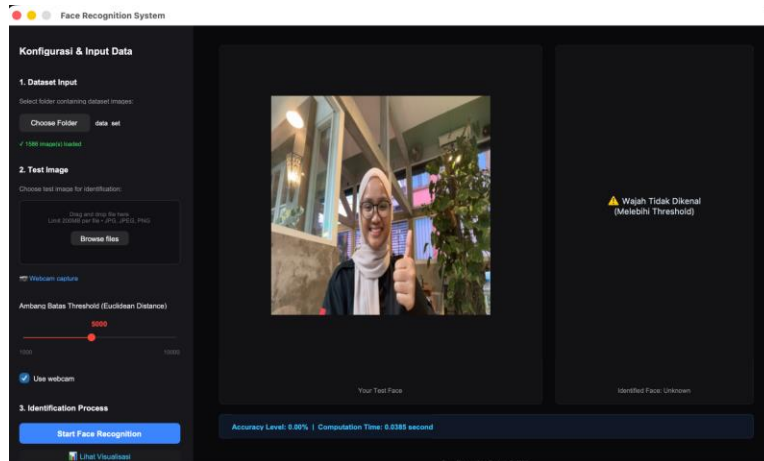
4.5. Tampilan aplikasi *face recognition* ketika gambar uji tidak ditemukan dalam *folder dataset*



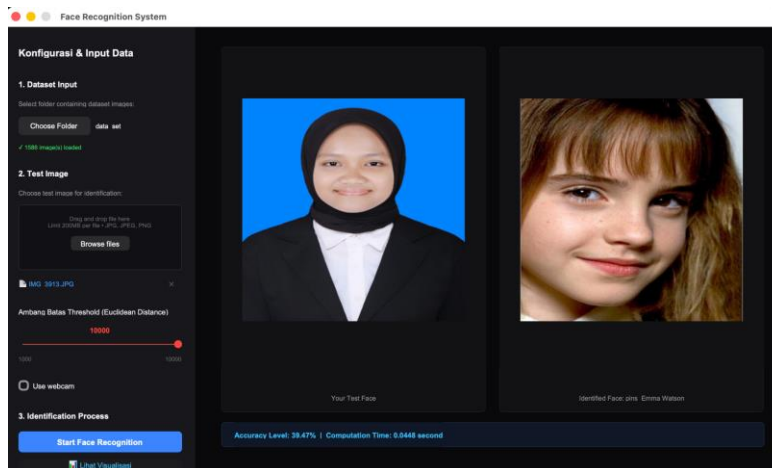
4.6. Tampilan aplikasi *face recognition* ketika gambar uji tidak ditemukan dalam *folder dataset*



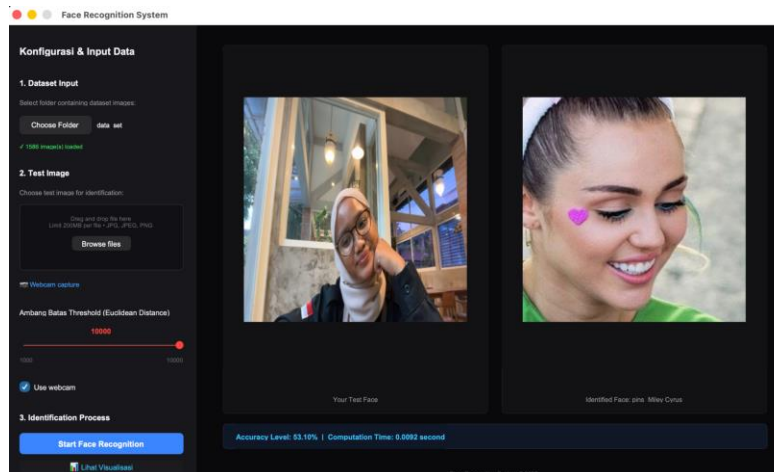
4.7. Tampilan aplikasi *face recognition* ketika menggunakan *webcam*



4.8. Tampilan aplikasi *face recognition* ketika hasil foto dari *webcam* tidak ditemukan dalam *folder dataset*



4.9. Tampilan aplikasi *face recognition* ketika menguji salah satu gambar yang tidak terdapat di *folder dataset* dengan *threshold* tertinggi



4.10. Tampilan aplikasi *face recognition* ketika menguji salah satu gambar yang tidak terdapat di *folder dataset* dengan *threshold* tertinggi

Analisis Hasil Percobaan

Sebelum melakukan eksperimen, aplikasi *face recognition* ketika pertama kali dibuka akan menghasilkan gambar 4.1. Selanjutnya, dilakukan eksperimen yang pertama, yakni memasukkan gambar uji berupa wajah individu yang gambarnya sudah ada di dalam *database*, seperti yang ditampilkan pada gambar 4.2 dan 4.3. Hasil dari pengujian pertama dan kedua ini menunjukkan bahwa sistem mampu mengenali gambar uji dengan baik. Nilai pada tingkat akurasi yang dihasilkan adalah 100%, menandakan bahwa sistem berhasil menemukan kemiripan antara gambar uji dan data yang sudah ada. Selanjutnya, dilakukan eksperimen yang kedua, yakni memasukkan gambar uji berupa wajah individu yang gambarnya tidak ada di dalam *database*, seperti yang ditampilkan 4.4. Hasil dari pengujian ketiga ini menunjukkan bahwa sistem tidak dapat mengenali gambar uji, meskipun gambar wajah individu tersebut dengan versi lain tersedia di dalam *database*. Hal ini terjadi dikarenakan gambar uji tersebut tidak ada dalam *database*, sehingga tidak dapat terdeteksi oleh sistem.

Dilakukannya eksperimen yang ketiga, yakni memasukkan gambar uji berupa wajah individu yang gambarnya tidak ada di dalam *database*, seperti yang ditampilkan 4.5. dan 4.6. Hasil dari pengujian keempat dan kelima ini menunjukkan bahwa sistem tidak dapat mengenali gambar uji, karena gambar dengan wajah individu tersebut tidak ada di dalam *database*. Nilai pada tingkat akurasi yang dihasilkan adalah 0%, menandakan bahwa sistem benar-benar tidak menemukan

gambar wajah individu yang sesuai dengan *database*. Pada aplikasi *face recognition* ini, juga terdapat fitur *webcam* seperti yang ditampilkan oleh gambar 4.7.

Dilakukan eksperimen yang keempat, yakni memasukkan foto uji hasil menggunakan *webcam* yang gambarnya tidak ada di dalam *database*, seperti yang ditampilkan 4.8. Hasil dari pengujian keenam ini menunjukkan bahwa sistem tidak dapat mengenali foto uji, karena gambar dengan wajah individu tersebut tidak ada di dalam *database*. Selanjutnya, dilakukan eksperimen yang terakhir, yakni memasukkan gambar uji yang tidak ada di dalam *database* sekaligus mengubah ambang batas *threshold* menjadi yang tertinggi, seperti yang ditampilkan oleh gambar 4.9 dan 4.10. Hasil dari pengujian ketujuh dan kedelapan ini menunjukkan bahwa sistem mendeteksi gambar uji tersebut memiliki kemiripan dengan gambar yang ada di dalam *database* dengan akurasi sebesar 39-53%.

BAB V

PENUTUP

Setelah melalui proses pengerjaan dan pengujian, program pengenalan wajah (*face recognition*) berbasis bahasa Python dengan algoritma *Eigenface* berhasil diimplementasikan sesuai dengan materi yang telah dipelajari dalam mata kuliah Aljabar Linear.

Program ini dikembangkan melalui dua tahap utama, yaitu tahap pelatihan (*training*) dan tahap pengenalan. Pada tahap pelatihan, seluruh gambar dalam *dataset* diubah ke dalam mode skala keabuan (*grayscale*), diratakan menjadi vektor satu dimensi, kemudian dihitung nilai rata-ratanya (*mean face*), lalu dinormalisasi dengan cara mengurangkan setiap gambar terhadap nilai rata-rata tersebut. Dari data yang telah dinormalisasi, dihitung matriks kovarians, yang selanjutnya digunakan untuk memperoleh nilai eigen dan vektor eigen secara manual dengan menerapkan metode *power iteration* dan deflasi. Vektor-vektor eigen inilah yang menjadi *eigenfaces* utama.

Setiap gambar kemudian diproyeksikan ke dalam ruang *eigenface* dan dihitung bobotnya (*weight*). Untuk proses pengenalan wajah, gambar uji dikurangi dengan *mean face* dan juga diproyeksikan ke dalam ruang *eigenface*. Bobot dari gambar uji selanjutnya dibandingkan dengan bobot-bobot dari *dataset* menggunakan perhitungan jarak Euclidean. Apabila jarak minimum yang dihasilkan lebih kecil dari nilai ambang batas tertentu, maka wajah tersebut dianggap berhasil dikenali.

Hasil percobaan menunjukkan bahwa gambar yang berasal dari dalam *dataset* dapat dikenali dengan hasil akurasi yang tinggi, sementara gambar dari luar *dataset* tidak dapat dikenali dan menghasilkan akurasi sebesar 0%. Hal ini membuktikan bahwa algoritma yang diterapkan telah berfungsi sesuai dengan spesifikasi yang diharapkan dan menunjukkan kinerja yang baik dalam mengenali wajah berdasarkan fitur-fitur dominan menggunakan metode *eigenface*.

LAMPIRAN

- a. *Repository* program: <https://github.com/adhiif-al/Project-Face-Recognition>
- b. Tautan video demo: uns.id/videoPBL-ALIN

DAFTAR PUSTAKA

- 7 Nilai Eigen dan Vektor Eigen – Aljabar Linear. (2026). Aljabar Linear. <https://bookdown.org/dsciencelabs/aln/07-Nilai-Eigen-dan-Vektor-Eigen.html>
- Bagian, & Munir, R. (n.d.). Nilai Eigen dan Vektor Eigen Bahan kuliah IF2123 Aljabar Linier dan Geometri Seri bahan kuliah Algeo #18. <https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2020-2021/Algeo-18-Nilai-Eigen-dan-Vektor-Eigen-Bagian1.pdf>
- Byju's. (n.d.). Eigenvalues (Definition, Properties, Examples) | Eigenvectors. BYJUS. <https://byjus.com/maths/eigen-values/>
- Chrisnalia, A., Ali, E., Mardainis, M., & Rahmiati, R. (2022). New student drug tests at college using Principal Component Analysis method. *IT Journal Research and Development*, 6(2), 98-108. <https://journal.uir.ac.id/index.php/ITJRD/article/view/7583>
- Elsevier. (2025). Eigenface. ScienceDirect Topics. Retrieved May 21, 2026, from <https://www.sciencedirect.com/topics/computer-science/eigenface>
- Farhansyah, R. (n.d.). EigenFace-Recognition [Source code]. GitHub. <https://github.com/rifqifarhansyah/EigenFace-Recognition>
- Pratiwi, D. E., & Harjoko, A. (2013). Implementasi pengenalan wajah menggunakan PCA (Principal Component Analysis). *IJEIS (Indonesian Journal of Electronics and Instrumentation Systems)*, 3(2), 175-184. <https://doi.org/10.22146/ijeis.3892>
- Program Studi Teknik Informatika. (2024). *Analisis teoretis dekomposisi ruang eigen dalam pengenalan wajah menggunakan PCA* (Makalah IF2123 Aljabar Geometri). Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. [https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2024-2025/Makalah/Makalah-IF2123-Algeo-2024%20\(26\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2024-2025/Makalah/Makalah-IF2123-Algeo-2024%20(26).pdf)
- Turk, M. A., & Pentland, A. P. (1991). Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1), 71-86. <https://doi.org/10.1162/jocn.1991.3.1.71>